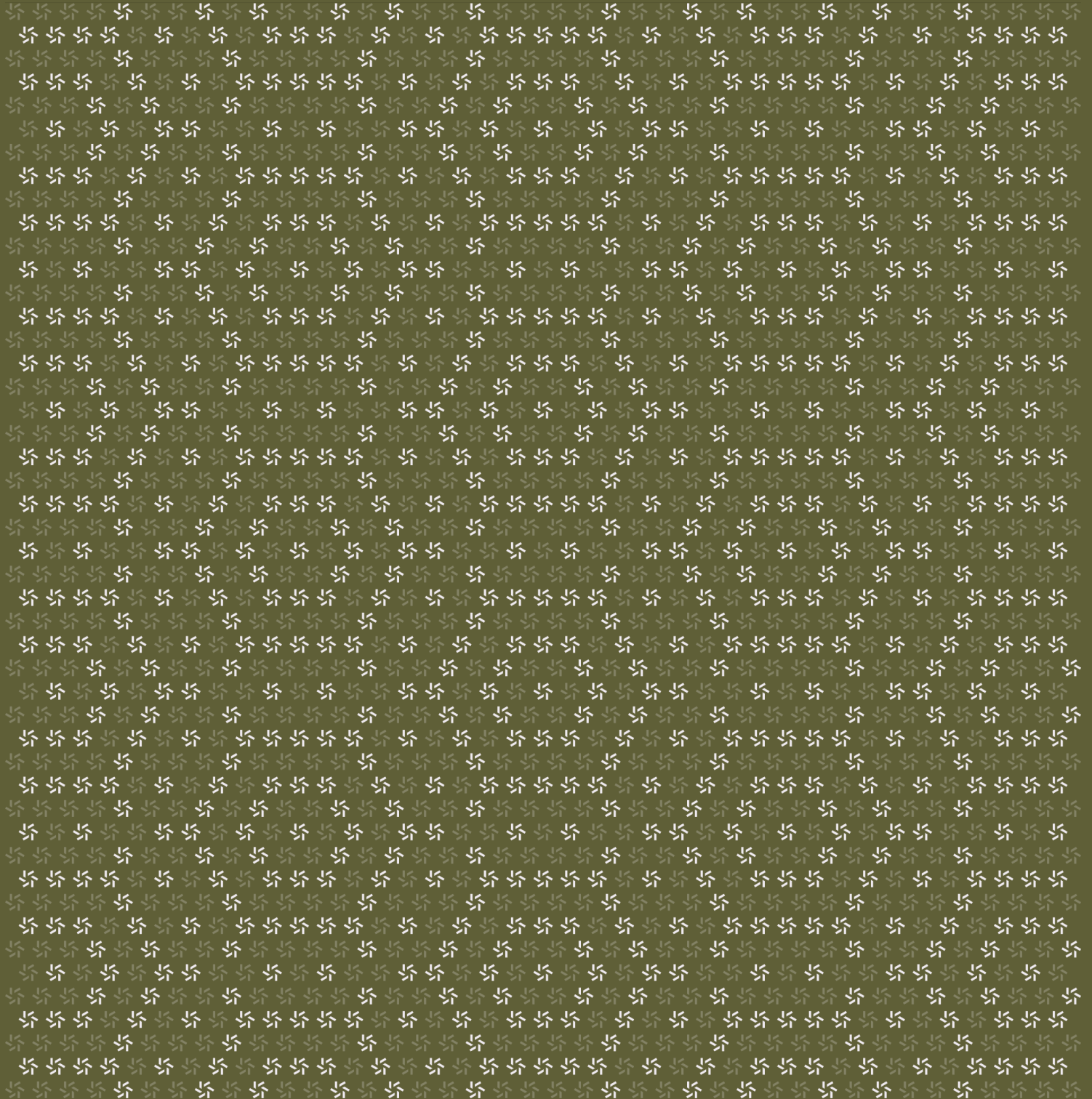


June 30, 2026

zecd

Application Security Assessment



Contents

About Zellic	5
1. Overview	6
1.1. Executive Summary	6
1.2. Goals of the Assessment	6
1.3. Non-goals and Limitations	6
1.4. Results	7
2. Introduction	8
2.1. About zecd	8
2.2. Methodology	8
2.3. Scope	10
2.4. Project Overview	10
2.5. Project Timeline	11
3. Detailed Findings	12
3.1. An upstream broadcast error without a numeric code is parsed as acceptance, returning a success txid for a rejected transaction	12
3.2. A send with an unbounded recipient count holds the single-writer wallet actor and blocks walletlock	15
3.3. Chain-status RPCs ignore the /wallet/<name> route and always report the default wallet	18
3.4. Stale block-cache metadata breaks reorg recovery and grows without bound	20
3.5. Unauthenticated requests bypass the RPC work queue	22

3.6.	The RPC cookie file is not forced to 0600 on a preexisting path or symlink, exposing a valid credential	24
3.7.	The identity file is not forced to 0600 on a preexisting path or symlink, exposing a valid credential	26
3.8.	Queued/executing <code>z_sendmany</code> jobs can grow the registry without bound since <code>MAX_OPERATIONS</code> only caps finished operations	28
3.9.	The zebra RPC backend is plaintext-only with no TLS option, exposing credentials and chain data to a remote node	31
3.10.	The <code>gettransaction</code> function overcredits a pure receive by the network fee and tags the receive with a fee field	33
3.11.	The <code>setlabel</code> function accumulates unbounded label storage and reloads the whole table on every history RPC	35
3.12.	The RPC password can be passed as a command-line argument, exposing it through process metadata	37
3.13.	Internal <code>anyhow::Error</code> messages are forwarded verbatim to the RPC client	39
3.14.	The cached proving-key path fails sends to Sapling recipients	41
3.15.	<code>/readyz</code> reports ready before the wallet has scanned	43
3.16.	The single-instance data-directory lock does not span hosts	45
4.	Discussion	46
4.1.	Initialization trusts the existing contents of the data directory	46
4.2.	The safe-by-default security posture is assumed rather than enforced	47
4.3.	The core-dump opt-out variable disables seed hardening for any value, not just the documented one	47
4.4.	A stalled upstream response can wedge synchronization indefinitely	48
4.5.	Reconnect backoff does not advance after a postconnection failure	49
4.6.	Unvalidated upstream responses can panic the scanner	49
4.7.	Through <code>sync.interval_secs = 0</code> , the idle delay between poll loops is removed	50

4.8.	The SIGTERM does not trigger the same graceful shutdown as Ctrl-C	51
5.	System Design	52
5.1.	Component: RPC server, authentication, and request admission	52
5.2.	Component: RPC dispatch and wallet method handlers	53
5.3.	Component: Wallet actor, key material, and storage	54
5.4.	Component: Chain source and the zebra backend	56
5.5.	Component: Background sync engine	57
6.	Assessment Results	59
6.1.	Disclaimer	59

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and follow @zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Overpay from June 18th to June 30th and July 1st to July 3rd, 2026. During this engagement, Zellic reviewed zecd's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Could a malicious or misbehaving chain backend (the upstream Zebra node) cause the wallet to derive an incorrect view of its funds, for example by injecting unconfirmed credits that are treated as real, failing to reverse credits across a chain reorganization, or halting the synchronization actor?
 - Are the JSON-RPC authentication and trust boundaries enforced correctly, and are the untrusted inputs that RPC clients forward on behalf of their own users (such as addresses, amounts, and memos) validated before they are acted upon?
 - Could a client be misled about the outcome of a spend, for example by being told that a transaction failed when it was in fact committed and relayed or that one succeeded when it was not?
 - Are the wallet's secrets, namely the mnemonic seed and the derived spending keys, protected at rest and in memory, including across the lock and unlock life cycle?
 - Does the wallet correctly attribute funds to unified and diversified addresses and report balances and received amounts consistently across the relevant RPC methods?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- All autogenerated files, tests, and mocks
- Repository files outside the in-scope `src/**/*.rs` daemon sources, including the regtest harness, deployment scripts, and build tooling
- Third-party dependencies and libraries (such as `librustzcash` and the upstream Zebra node), except where their usage by in-scope code introduces an issue
- The trusted-execution-environment (Nitro Enclave) and AWS-KMS keystore deployment variants, which were a work in progress at the time of review
- Front-end components
- Infrastructure relating to the project

- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

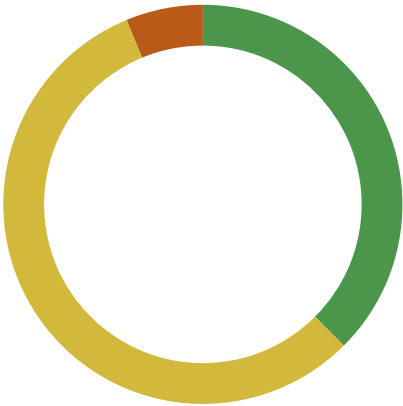
1.4. Results

During our assessment on the scoped zecd files, we discovered 16 findings. No critical issues were found. One finding was of high impact, nine were of medium impact, and six were of low impact.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Overpay in the Discussion section (4.7).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	1
Medium	9
Low	6
Informational	0



2. Introduction

2.1. About zecd

Overpay contributed the following description of zecd:

zecd is a shielded-only wallet server built on librustzcash, exposed through bitcoind's RPC dialect for developer ease of use. It uses the same method names, response shapes, auth, and error codes as Bitcoin Core, so that many existing Bitcoin RPC clients can use Zcash with little or no changes.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Unsafe code patterns. Many vulnerabilities in web-based applications arise from oversights during development. Missing an authentication check on a single API endpoint can critically impact the security of the overall application. Failure to properly sanitize and encode user input can lead to vulnerabilities like SQL injection, server-side request forgery, and more. We use both automated tools and extensive manual review to identify unsafe code patterns.

Business logic errors. Business logic is the heart of all web applications. We carefully review logic to ensure that the code securely and correctly implements the specified functionality. We also thoroughly examine all specifications for inconsistencies, flaws, and vulnerabilities, including for risks of fraud and abuse.

Integration risks. Web projects frequently have many third-party dependencies and interact with services and libraries that are not under the developer's control. We review the project's external interactions and identify potentially dangerous behavior resulting from compatibility issues and data inconsistencies.

Code maturity. We look for possible improvements across the entire codebase, reviewing violations of industry best practices and code quality standards. We suggest improvements to code clarity, documentation, and testing practices.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped files itself. These observations — found in the Discussion (4.7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

zecd Files

Type	Rust
Platform	Zcash
Target	Primary review
Repository	https://github.com/zecrocks/zecd ↗
Version	fca78b216c75a1b95e4be6ae2fd141ea4acae4c4
Programs	src/**/*.*rs
Target	Secondary review (v0.3.1) — changes between fca78b21 and 04d70f7
Repository	https://github.com/zecrocks/zecd ↗
Version	04d70f7c3147289e361f61b8f8e26c8ac1cb443f
Programs	src/**/*.*rs

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 2.7 person-weeks. The assessment was conducted by two consultants over the course of 1.8 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✂ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Jihwan Kim
✂ Engineer
jihwan@zellic.io ↗

Seokchan Yoon
✂ Engineer
seokchan@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

June 18, 2026	Kick-off call
----------------------	---------------

June 18, 2026	Start of primary review period
----------------------	--------------------------------

June 30, 2026	End of primary review period
----------------------	------------------------------

July 1, 2026	Start of secondary review period
---------------------	----------------------------------

July 3, 2026	End of secondary review period
---------------------	--------------------------------

3. Detailed Findings

3.1. An upstream broadcast error without a numeric code is parsed as acceptance, returning a success txid for a rejected transaction

Target	ChainSource, WalletActor		
Category	Coding Mistakes	Severity	High
Likelihood	Low	Impact	High

Description

zecd talks directly to a zebra over JSON-RPC and models the node's verdict on a broadcast as a `BroadcastOutcome`, whose `is_accepted()` is defined purely as `error_code == 0`. The send path commits the transaction to the `WalletDb` first, then broadcasts it, and returns the txid to the caller whenever the broadcast is accepted. Under bitcoin's contract, a returned txid means the network has the transaction, so `error_code` is the single load-bearing signal, and it doubles as both the "no error" default and the success sentinel.

```
// src/chain/mod.rs:53
pub fn is_accepted(&self) -> bool {
    self.error_code == 0
}
```

When zecd parses the zebra error envelope, it defaults a missing or noninteger code to 0:

```
// src/chain/zebra.rs:194
if let Some(err) = envelope.get("error").filter(|e| !e.is_null()) {
    return Err(CallError::Rpc {
        code: err.get("code").and_then(Value::as_i64).unwrap_or(0), //
        // missing/non-integer code -> 0
        // [...]
    });
}
```

Then `broadcast_tx` turns that `CallError::Rpc` into an `Ok(BroadcastOutcome { .. })` rather than a transport error, carrying the parsed code straight through,

```
// src/chain/zebra.rs:464
Err(CallError::Rpc { code, message }) => Ok(BroadcastOutcome {
    error_code: i32::try_from(code).unwrap_or(i32::MIN),
    error_message: message,
}),
```

and `broadcast_committed` only enters its rejection branch (the `-26` path) on `!outcome.is_accepted()`, so a codeless rejection falls through to the success return and the `txid` is handed back to the caller.

```
// src/wallet/actor.rs:1188
if !outcome.is_accepted() {
    // [...] codes::RPC_VERIFY_REJECTED (-26)
}
Ok(())
```

Impact

A returned `txid` is the client's proof that the network accepted the transaction. Because a codeless rejection collapses to `error_code == 0`, a transaction that the upstream node actually rejected is reported to the caller as a successful send. The transaction was never relayed, so the wallet's input notes stay locked until expiry while the client acts on a `txid` the network never saw. A merchant or marketplace that treats the returned `txid` as settlement can release goods or credit an account against a payment that does not exist.

The trigger is narrow, requiring a valid JSON error object that carries a message but no numeric code; non-JSON responses are already classified as transport errors. It is most reachable in containerized or Kubernetes deployments where a proxy, load balancer, or service mesh sits in front of zebra and can emit its own error body without a JSON-RPC code.

Concretely, a rejected send surfaces to the caller as a success through the following sequence:

1. A client calls `sendtoaddress` or `z_sendmany`; zecd commits the transaction to the `WalletDb` and broadcasts it.
2. The upstream zebra, or a proxy in front of it, returns a JSON-RPC error envelope with a message but no numeric code field.
3. The `unwrap_or(0)` default makes that rejection parse as `error_code == 0`.
4. Then `is_accepted()` is true, `broadcast_committed` returns the `txid`, and the caller is told the send succeeded.
5. The transaction was never relayed, and the client acts on a `txid` the network never saw.

Recommendations

Separate the parsing default from the success sentinel. A `CallError::Rpc` should always be classified as a rejection regardless of whether a numeric code is present, so that the accepted state is only reachable on the path where zebra actually returns success. Concretely, map a missing or noninteger code to an explicit nonzero rejection code rather than to 0, and add a regression test asserting that a codeless error envelope is treated as a rejection.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [1a1c764](#).

3.2. A send with an unbounded recipient count holds the single-writer wallet actor and blocks walletlock

Target	WalletActor, wallet RPC methods		
Category	Coding Mistakes	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

Each wallet is a single-writer actor that processes one command at a time, which is what gives zecd its no-double-spend guarantee. The cost of that design is serialization. While one command is running, every other command for that wallet waits its turn behind it, so a single slow command stalls all the others. The `sendtoaddress`, `sendmany`, and `z_sendmany` handlers build the Sapling and Orchard proofs in-line inside the actor under `tokio::task::block_in_place`, and proving cost grows roughly linearly with the number of output actions, which is the number of recipients. So the time the actor is held is set by how many recipients the caller puts in one send.

There is no cap on recipient count anywhere in zecd; `sendmany` walks the whole address-to-amount object and pushes one payment per entry, with only an is-empty check, and the only ceiling is ZIP-321's 9999 inside `librustzcash`.

```
// src/rpc/wallet_methods.rs:1391
let mut payments = Vec::new();
for (addr, amount) in recipients {
    payments.push(build_payment(/* [...] */)?);
}
// [...] only an is-empty check; no recipient-count limit
(TransactionRequest::new enforces only ZIP-321's 9999)
let request = TransactionRequest::new(payments)?;
```

First, `do_send` checks funds via `propose_transfer` and then proves via `create_proposed_transactions`, both inside the `block_in_place` scope on the actor thread, and nothing wraps the proving in a time-out.

```
// src/wallet/actor.rs:1072
tokio::task::block_in_place(move || -> Result<_, RpcError> {
    let proposal = propose_transfer(/* [...] */)?;
    let txids = create_proposed_transactions(/* [...] */)?; // one zk proof
    per output
    // [...]
})?;
```

There is no fast path for `walletlock`. It is delivered as a `WalletCommand::Lock` on the same bounded command channel, so it waits its turn behind whatever the actor is already running and cannot interrupt a send that is midproof.

```
// src/wallet/actor.rs:983
WalletCommand::Lock { reply } => { /* [...] */ } // waits behind an
in-flight send
```

Impact

A single authenticated `sendmany` or `z_sendmany` with a few thousand recipients holds the wallet actor for the entire proving window, which can be minutes to hours. For that whole window, every other command on that wallet is parked behind it, including other sends, `getnewaddress`, `walletpassphrase`, and `walletlock`. Because legitimate users of the same wallet are blocked too, this is not a pure self-DOS.

Most importantly, the operator cannot lock the wallet: the `walletlock` command sits unprocessed until the giant send finishes, so the decrypted seed stays resident for an attacker-chosen duration, defeating the emergency-lock control. A small request (a few kilobytes of JSON) forces minutes to hours of ZK-proving CPU on the server.

The magnitude is bounded by funds, since `propose_transfer` fails with `-6` before any proving when the balance cannot cover the outputs plus the per-output ZIP-317 fee. So the practical wedge length is the number of outputs the wallet can actually fund (roughly 0.05 ZEC reaches about 1,000 outputs, recoverable via self-payment). And `z_sendmany` makes it cheaper to land, since it returns an opid immediately and runs the send on a detached task through the same actor, so the actor wedges identically while the caller's connection and work-queue permit are never tied up.

A single authenticated request wedges the actor like this:

1. An operator runs an unlocked, funded wallet serving RPC.
2. An authenticated caller (or a subuser whose batch payout the operator forwards) sends `sendmany` or `z_sendmany` with a few thousand distinct recipients and small amounts the wallet can fund.
3. The `propose_transfer` succeeds, so `do_send` proceeds into `create_proposed_transactions` and proves one transaction with that many output actions, holding the actor under `block_in_place`.
4. For the whole proving window, every other command on that wallet is parked, including `walletlock`.
5. The operator calls `walletlock`; the command waits behind the send, so the wallet cannot be locked and the decrypted seed stays resident.

Recommendations

Enforce a zecd-side cap on the recipient count per send, well below ZIP-321's 9999 and matched to operational expectations, rejecting oversized requests at the `sendmany`, `z_sendmany`, and `sendtoaddress` entry points before any proving. Reject multitransaction proposals before proving rather than after, so inputs that will be rejected anyway do not consume proving CPU. Consider moving long proving onto a dedicated worker outside the actor's serialization while preserving the double-spend serialization invariant, so that security commands such as `walletlock` can be serviced while a send is in progress.

Remediation

This issue has been acknowledged by Overpay, and fixes were implemented in the following commits:

- [be46497](#) ↗
- [2663aa4](#) ↗
- [91ce52c](#) ↗

3.3. Chain-status RPCs ignore the /wallet/<name> route and always report the default wallet

Target	RPC dispatch, blockchain handlers		
Category	Coding Mistakes	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

zecd is multiwallet, with one spending wallet plus any number of watch-only replicas, each addressed as /wallet/<name>. Wallet methods such as getbalance honor that route, but the five chain-status RPCs (getblockchaininfo, getblockcount, getbestblockhash, getblockhash, and getblockheader) do not. Each resolves registry.get(None), which is the default wallet, and dispatch never forwards the routed name to them. So a caller polling a lagging nondefault wallet is told the default wallet's height and sync state instead of its own.

The handler reads the default wallet regardless of the route:

```
// src/rpc/blockchain.rs:55
pub(crate) fn getblockcount(state: &AppState) -> Result<Value, RpcError> {
    let w = state.registry.get(None)?; // always the default
    wallet
    Ok(json!(w.status().fully_scanned.unwrap_or(0)))
}
```

Dispatch calls that arm with state only, dropping the routed wallet that the wallet methods receive:

```
// src/rpc/mod.rs:120
"getblockchaininfo" => blockchain::getblockchaininfo(state),
"getblockcount"     => blockchain::getblockcount(state), // no wallet
                      argument
```

Impact

In a multiwallet deployment, a nondefault payment wallet that is still scanning is reported with the default wallet's caught-up height and sync state. An integration that computes confirmations from getblockcount, or that reads getblockchaininfo.initialblockdownload to decide that synchronization is complete, is therefore told the wrong wallet's status. It can treat a deposit that has not yet been scanned on the routed wallet as confirmed and release goods or credit an

account early. The polling pattern recommended in zecd's own documentation, `getblockhash(getblockcount())`, also operates against the default wallet when issued on a nondefault route. No funds leak directly, but the chain-status answer is decoupled from the routed wallet, which misleads confirmation logic in automation.

Concretely, the misreport unfolds as follows:

1. The operator runs a default wallet that is caught up and a payment wallet `w2` that is still scanning, say 1,500 blocks behind.
2. An integration polls `/wallet/w2` with `getblockcount` (or checks `getblockchaininfo.initialblockdownload`) to decide whether a deposit has enough confirmations.
3. Dispatch routes `/wallet/w2` to `getblockcount`, which reads `registry.get(None)`, the default wallet.
4. It returns the default wallet's height and `initialblockdownload=false`, even though `w2` is 1,500 blocks behind and its balances are stale.
5. The integration treats a not-yet-scanned deposit on `w2` as confirmed and releases goods or credits early.

Recommendations

Have dispatch forward the routed `wallet` to the blockchain handlers as it already does for wallet methods, and have each handler read `registry.get(wallet)` rather than `registry.get(None)`. The best-block and median-time-past helpers should resolve the same routed wallet so that all chain-status answers are consistent with the wallet named in the route.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [ca5a300](#).

3.4. Stale block-cache metadata breaks reorg recovery and grows without bound

Target	Sync engine		
Category	Coding Mistakes	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

During forward sync, `sync_one_batch` deletes the compact-block files after scanning each batch, but it never removes the matching `compactblocks_meta` rows in the `FsBlockDb`. So in a caught-up node, the metadata rows remain for every scanned height while the files are gone, and the on-disk cache and the metadata table drift apart — `delete_cached_blocks` removes only the files:

```
// src/sync/engine.rs:140
fn delete_cached_blocks(wallet_dir: &Path, block_meta: Vec<BlockMeta>) {
    for meta in block_meta {
        if let Err(e) = std::fs::remove_file(block_path(wallet_dir, &meta)) {
            /* warn */ }
        }
    }
}
```

On a reorg, the rewind reads the cache before it truncates it, and that read opens every retained metadata row's file,

```
// src/sync/engine.rs:242
db_cache
    .with_blocks(Some(rewind_height + 1), None, |block| { /* [...]
        */ })?; // errors on a deleted file
db_cache.truncate_to_height(rewind_height)?; //
        never reached
```

and `with_blocks` opens each metadata row's backing file with `File::open`, which fails with `NotFound` for an already deleted file. The error propagates out of `with_blocks` before `truncate_to_height` runs.

```
// zcash_client_sqlite-0.21.0/src/chain.rs:301
File::open(cbr.block_file_path(&cache.blocks_dir)).map_err(to_chain_error)?;
```

Impact

This is triggered by a naturally occurring reorg, with no attacker required, after normal multibatch forward sync. The designed in-place rewind errors out on every such reorg, so the actor can only recover by dropping the client and redownloading, degrading recovery quality. Because `with_blocks` fails before `truncate_to_height`, the cache metadata is never truncated either, so `compactblocks_meta` accumulates without bound over the life of the node, growing disk usage and query cost. There is no loss of funds and the node self-heals via redownload, but the only intended reorg-recovery path does not work as designed and is paired with unbounded growth. This is especially relevant around the Ironwood cutover, when reorgs may be more frequent.

The failure branch was reproduced with a unit test (`poc_reorg_fails_when_prior_batch_files_deleted`) that drives the real `scan_blocks`, `delete_cached_blocks`, and an on-disk `FsBlockDb`: scanning heights 1 through 10 in batches with per-batch file deletion, then presenting a reorg, makes `scan_blocks` fail and leaves the cache metadata untruncated.

```
PoC: reorg in-place rewind failed as expected: BlockSource(Fs(0s { code: 2,
    kind: NotFound, ... }))
test ... ok
```

Recommendations

Keep the cached files and the metadata rows consistent: have `delete_cached_blocks` remove the matching `compactblocks_meta` rows along with the files, or reorder reorg recovery so that `truncate_to_height` (which clears the metadata rows) runs before `with_blocks` (which opens the files). Alternatively, treat a missing file inside `with_blocks` as already cleaned rather than a fatal error.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [3ef3604](#).

3.5. Unauthenticated requests bypass the RPC work queue

Target	RPC server		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

zecd's work queue is meant to bound in-flight requests, returning 503 like bitcoind once the configured depth is exceeded. However, the authentication check and its 250ms anti-brute-force sleep run before the work-queue gate, so unauthenticated traffic is never counted against the queue. A bad-credential request hits the sleep and the 401 and returns before any permit is taken:

```
// src/server/mod.rs:81
if !state.auth.check(auth_header) {
    tokio::time::sleep(std::time::Duration::from_millis(250)).await;
    return unauthorized();
}
// src/server/mod.rs:88 (permit acquired only after auth succeeds)
let _permit = match state.work_queue.clone().try_acquire_owned() {
    Ok(p) => p,
    Err(_) => return plain_response(StatusCode::SERVICE_UNAVAILABLE, "Work
queue depth exceeded"),
};
```

Impact

The in-flight bound (the work_queue semaphore) is acquired only after authentication succeeds, so the documented overload protection does not apply to unauthenticated traffic. An attacker who can reach the RPC port, with no credentials, can flood bad-credential requests; each one runs the auth check, sleeps 250 ms, and returns 401, holding a connection and a timer slot but never taking a work-queue permit. The configured in-flight bound never caps that traffic the way it caps authenticated load, so legitimate clients see degraded availability. There is no loss of funds or data; this is an availability issue, but it is reachable preauthentication, so its reach is broad in any deployment where the RPC port is exposed beyond the trust boundary.

This was verified with a test against the real router with the work queue fully exhausted (zero permits); an authenticated request gets 503 as expected, but a bad-credential request still gets 401, which proves the queue does not bound unauthenticated traffic.

An attacker with no credentials degrades availability through this loop:

1. An unauthenticated attacker reaches the RPC port, with no credentials needed.
2. They flood bad-credential requests.
3. Each request runs the auth check, sleeps 250 ms, and returns 401, holding a connection and a timer slot but never taking a work-queue permit.
4. The configured in-flight bound never applies to that traffic, so the flood is not capped the way authenticated load is, and legitimate clients see degraded availability.

Recommendations

Apply admission control (the work-queue permit or a separate cheap in-flight counter) before the authentication check and its 250ms sleep, so that unauthenticated requests are included in the in-flight bound. Alternatively, bound the unauthenticated path itself by limiting the number of concurrent auth-failure sleeps. Both belong to the same class of preauthentication resource consumption, where the server expends effort on a request before it is authenticated.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [ca5a300](#).

3.6. The RPC cookie file is not forced to 0600 on a preexisting path or symlink, exposing a valid credential

Target	RPC cookie authentication		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

When no [rpc] user, password, or auth is set, zecd falls back to cookie auth like bitcoind. It writes `__cookie__:<random> to <datadir>/ .cookie`, and that line is a full Basic credential. The write is meant to be 0600, but the way it is done only enforces that mode when the file is freshly created. The `write_cookie` function opens the path with `create(true).truncate(true).mode(0o600)`. On Unix, `.mode(0o600)` is the creation mode, so it is ignored when the file already exists, and `create(true)` (not `create_new`) happily opens and truncates an existing file. There is also no `O_NOFOLLOW`, so the open follows a symlink planted at that path.

```
// src/server/auth.rs:198
fn write_cookie(path: &Path, user: &str, password: &str) ->
    std::io::Result<> {
    // [...]
    let mut opts = std::fs::OpenOptions::new();
    opts.write(true).create(true).truncate(true); // create(true) !=
    create_new, opens an existing file
    #[cfg(unix)]
    {
        use std::os::unix::fs::OpenOptionsExt;
        opts.mode(0o600); // applied only on
        create, ignored for an existing file
    }
    let mut file = opts.open(path)?; // no O_NOFOLLOW, follows
    a symlink
    file.write_all(format!("{user}:{password}").as_bytes())?;
    Ok(())
}
```

The comment directly above this code claims the cookie is created 0600 atomically and is never briefly world-readable, which is true only for the fresh-create path.

Impact

The cookie contents `__cookie__`:<random> are a complete RPC credential that passes `Authenticator::check`, and an authenticated RPC client can read balances, addresses, and memos and, on an unlocked wallet, send. Because the mode and the symlink handling are enforced only on fresh creation, two cases expose the credential on a shared host:

1. If a world-readable file already sits at the cookie path (a stale `.cookie` from a prior run, or one left by a permissive umask), zecd writes the credential into it and leaves the mode at whatever it already was (for example, 0644), so another local user can read a valid RPC credential.
2. If an unprivileged user plants a symlink at the cookie path before zecd starts, the open follows it, so the credential is written to a location the attacker can read or used to overwrite an arbitrary file.

This was confirmed with a test against the real `write_cookie`; a preexisting 0644 file keeps 0644 with the credential sitting in cleartext. The trigger is local and same-host, which is why this is not higher severity, but the result is exposure of a valid RPC credential, and the cookie path is the default fallback. This is a different file from the age identity file whose permissions were addressed separately.

Recommendations

Follow bitcoind's pattern: write the credential to a temporary file created with `create_new(true)` and mode 0600, then rename it into place atomically, rather than opening the fixed path with `create(true)`. On Unix, set `O_NOFOLLOW` to refuse a symlink at the path, and `fchmod` the open descriptor to 0600 so the mode is enforced even for an existing file. The shutdown-time cookie removal is already handled, so only the creation invariant needs hardening.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [cfe3dea7](#).

3.7. The identity file is not forced to 0600 on a preexisting path or symlink, exposing a valid credential

Target	Wallet startup		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

While the wallet is launching, zecd writes the recipient's public key and identity secret into a file. This file is created with permission 0600, restricting read and write to the owner, which keeps the secret confidential at the first level. However, when the identity file already exists, zecd loads it without verifying its permission mode. Additionally, the load path follows a symlink at that path instead of requiring a regular file (the create path is symlink-safe, since it uses `create_new`).

```
// src/init.rs:384

async fn ensure_identity(path: &Path) ->
    anyhow::Result<Vec<Box<dyn age::Recipient + Send>>> {
    if tokio::fs::try_exists(path).await.unwrap_or(false) {
        /* Missing permission check when file exists */
        let recipients =

age::IdentityFile::from_file(path.to_string_lossy().into_owned())?.to_recipients()
        return Ok(recipients);
    }

    // [code goes on]

    let mut opts = tokio::fs::OpenOptions::new();
    opts.write(true).create_new(true);
    #[cfg(unix)]
    {
        // tokio's OpenOptions exposes `mode` as an inherent method (no trait
        import needed).
        opts.mode(0o600); /* Only applied while creation, not load */
    }
    let mut f = opts.open(path).await?;
    f.write_all(b"# zecd age identity (KEEP SECRET)\n").await?;
    f.write_all(format!("# public key: {recipient}\n").as_bytes())
        .await?;
```

```
f.write_all(format!("{}",\n",
identity.to_string().expose_secret()).as_bytes())
    .await?;
f.flush().await?;
```

Impact

If the identity file already exists with weak permissions, any other local user can read it and obtain the age identity secret. That secret is the key that decrypts the wallet's stored mnemonic, but the mnemonic itself lives in `keys.toml`, which `zecd` always writes mode `0600`, so reading the identity alone yields the decryption key without the ciphertext it applies to. An attacker who can also read `keys.toml`, for example if both files were restored or created with permissive modes, can then decrypt the mnemonic, derive the seed, and compromise the wallet fully. On its own, the exposure is a defense-in-depth weakness in one of the two secrets that guard the seed. The symlink case is weaker still, since a foreign identity planted at the path does not decrypt the operator's own `keys.toml`. The trigger is a low-complexity, non-root local read on a shared host or container.

Recommendations

Check the permissions on the existing identity file before loading it, and reject or warn if they are not strictly `0600` — the same as OpenSSH does for key files. Also reject symlinks, and accept only a regular file. On Unix, `std::fs::symlink_metadata` can be used to check both the mode and the file type before touching the secret. Fail closed if the permissions are too open, rather than loading the secret regardless.

Remediation

This issue has been acknowledged by Overpay, and fixes were implemented in the following commits:

- [7026e57](#) ↗
- [5651627](#) ↗

3.8. Queued/executing z_sendmany jobs can grow the registry without bound since MAX_OPERATIONS only caps finished operations

Target	Wallet RPC methods (z_sendmany)		
Category	Coding Mistakes	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

The z_sendmany function does not send synchronously. It spawns the send on a background task and returns an opid right away, while the actual AsyncOperation is tracked in OperationRegistry.

```
// src/rpc/wallet_methods.rs:1571

// Spawn the send on a background task and return the opid immediately. Every
// send error
// (including `-6` insufficient funds) surfaces later via the operation's
// `error`, never as
// a synchronous error here. The send still funnels through the single-writer
// actor, so the
// no-double-spend invariant holds exactly as for the synchronous sends.
let send_handle = handle.clone();
let op = AsyncOperation::new(Some(context), async move {
    let txid = send_handle.send(request, Some(policy), privacy).await?;
    Ok:::<Value, RpcError>(json!({ "txid": txid.to_string() })))
});
let opid = state.operations.insert(&handle.name, op); /* This code assumes
evict_if_over_cap here */
Ok(Value:::<String>(opid))
```

insert() is supposed to be bounded by MAX_OPERATIONS, which is enforced in evict_if_over_cap.

```
// src/operations.rs:27-30, 358-380

/// Cap on retained operations. On overflow the oldest *finished* operations
/// are evicted, so a
/// client that fires `z_sendmany` and never reaps results cannot grow the
/// registry without
/// bound. A queued or executing operation is never evicted (its result is
/// still pending).
```

```
const MAX_OPERATIONS: usize = 1024;

// ...

/// Evict the oldest finished operations once the map exceeds
/// [MAX_OPERATIONS]. Never
/// evicts a queued or executing operation.
fn evict_if_over_cap(map: &mut HashMap<OperationId, OperationEntry>) {
    if map.len() <= MAX_OPERATIONS {
        return;
    }
    let mut finished: Vec<(SystemTime, OperationId)> = map
        .iter()
        .filter(|(_, e)| e.op.state().is_finished()) /* Only evicts finished
        async job */
        .map(|(id, e)| (e.op.creation_time, id.clone()))
        .collect();
    finished.sort_by_key(|(t, _)| *t);
    let excess = map.len() - MAX_OPERATIONS;
    for (_, id) in finished.into_iter().take(excess) {
        map.remove(&id);
    }
}
```

Since `evict_if_over_cap` only ever removes `is_finished()` entries (Cancelled | Failed | Success), it does nothing at all if every entry in the map is still Ready / Executing.

Impact

An attacker who can call `z_sendmany` can register far more than `MAX_OPERATIONS` entries in the registry. Because every send still funnels through the single-writer actor, at most one send is proven at a time, so this is not CPU exhaustion. The pressure is on memory instead. Each submitted operation adds a registry entry that `evict_if_over_cap` never reclaims while it is still queued or executing, and it also holds a live spawned task that captures the full send request (all recipients, the policy, and the privacy setting) until that task drains through the serial actor and completes.

Submitting an operation is cheap, since `z_sendmany` returns the `opid` immediately and releases the connection and its work-queue permit, so an attacker can enqueue operations far faster than the actor can drain them. The registry entries and the backlog of pending task state therefore accumulate without bound. The end state is unbounded memory growth that drives the process toward memory exhaustion and an OOM kill, at which point the daemon is terminated by the OS or becomes unstable and stops serving requests. The sustained effect is memory-driven daemon instability rather than CPU saturation.

Recommendations

Set a limit on queued operations. The simplest fix would be to track both the queued and executing counts and reject a new `insert()` (or let `z_sendmany` return an error) once that count reaches the limit. Alternatively, the cap could be split so that each state — queued, executing, and finished — has its own budget.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [40a0194](#).

3.9. The zebra RPC backend is plaintext-only with no TLS option, exposing credentials and chain data to a remote node

Target	ZebraClient		
Category	Code Maturity	Severity	Low
Likelihood	Low	Impact	Low

Description

In the zebra-only backend, zecd is a light client that trusts a zebra JSON-RPC node for everything across the node boundary: blocks, tree state, the synthesized mempool, and getrawtransaction lookups. The JSON-RPC client carries no transport encryption of its own and is conventionally pinned to localhost, so the moment that node is remote, the Authorization header (the RPC user:password, or the contents of the cookie file) and every block and transaction travel in cleartext. The client only ever builds a plaintext http:// connection, with no https path, no ClientTlsConfig, and no loopback check on the configured host.

```
// src/chain/zebra.rs:149
pub fn new(host: &str, port: u16, auth: &ZebraAuth) ->
    anyhow::Result<ZebraClient> {
    let url: hyper::Uri = format!("http://{host}:{port}/").parse()?; //
    http:// hardcoded
    Ok(ZebraClient {
        http:
        HyperClient::builder(TokioExecutor::new()).build(HttpConnector::new()), //
        plaintext connector
        url,
        auth_header: auth.header()?, // Basic base64(user:password) or
        cookie
    })
}
```

The credential is attached to every request that goes out over that plaintext connection.

```
// src/chain/zebra.rs:166
if let Some(auth) = &self.auth_header {
    req = req.header(hyper::header::AUTHORIZATION, auth.clone());
}
```

The zebra://host:port in the config is only the backend selector; the resolver strips the scheme

and hands the bare `host:port` to the plaintext client above. The default is the zebra preset (127.0.0.1), so a colocated node is fine, but a remote host gets that same cleartext connection and nothing downgrades or refuses it.

Impact

In the default deployment, where zebra is on localhost, the plaintext loopback connection carries no real risk, consistent with how bitcoind, zcashd, and zebra RPC are conventionally plaintext on localhost. However, if an operator configures a remote zebra (a shared node on another host, or one across an internal network), the RPC credential travels in cleartext and can be recovered by passive eavesdropping, letting an attacker take over the node RPC, and every block and transaction can be tampered with by an active man-in-the-middle.

The previous lightwalletd path validated TLS, so removing it in the zebra-only backend is a transport-security regression. The default is localhost and a remote node is the operator's choice, which is why this is Low, but in a deployment that supports and uses a remote zebra, the cleartext credential exposure is real.

This was reproduced against the real ZebraSource and ZebraClient driving a fake zebra: the Authorization header observed on the wire Base64-decodes back to the configured `user:password`, and a fake node can substitute attacker bytes for a queried txid's `getrawtransaction` response, which `fetch_tx` returns without verification.

Recommendations

Gate on the transport rather than on the address. A strict `is_loopback` check is too restrictive, since the client requires RFC 1918 private addresses for Kubernetes (where the pod overlay already encrypts traffic). For a non-loopback host, require either TLS (`https` with certificate validation) or an explicit operator opt-out, and otherwise refuse to start. Provide a clearly named opt-out flag (for example, a `dangerous-no-tls` mode) for operators on an encrypted cluster, and default remote-plus-plaintext to refusal.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [47674b3](#).

3.10. The gettransaction function overcredits a pure receive by the network fee and tags the receive with a fee field

Target	Wallet RPC methods (gettransaction)		
Category	Coding Mistakes	Severity	Low
Likelihood	High	Impact	Low

Description

The gettransaction function builds its top-level amount as `account_balance_delta + fee_paid`, applied the same way to sends and receives. The code assumes `fee_paid` is set only when the wallet funded the transaction, so a pure receive would have it as `None` and amount would equal the received value.

```
// src/rpc/wallet_methods.rs:491
fn gettransaction_amount(account_balance_delta: i64, fee_paid: Option<u64>)
-> i64 {
    // assumes a pure receive has fee_paid == None; in fact a shielded receive
    has Some(F)
    account_balance_delta + fee_paid.map(|f| f as i64).unwrap_or(0)
}
```

That assumption is incorrect; `fee_paid` is `v_transactions.fee_paid`, which is `transactions.fee`, and `librustzcash` sets that for any fully shielded transaction, derived from the public Sapling and Orchard `valueBalance`, regardless of who created it. `zezd`'s receives are all fully shielded, so a pure receive lands with `account_balance_delta = +N` and `fee_paid = Some(F)`, and amount comes out as `N + F`. The same `fee_paid` then attaches a `fee = -F` field that bitcoind only ever puts on sent transactions.

```
// src/rpc/wallet_methods.rs:1043
if let Some(fee) = rec.fee_paid {
    obj["fee"] = signed_zats_to_value(-(fee as i64));
}
```

Impact

A merchant or exchange that reads `gettransaction(<receive_txid>).amount` to confirm that `N` arrived gets `N + F` instead of `N`, overcrediting by the network fee on every deposit, and the returned object also carries a `fee = -F` that breaks bitcoind conformance, since Core only reports

a fee on sent transactions. The per-transaction error is exactly the small ZIP-317 fee, so this is an accounting-integrity and conformance issue rather than a theft vector, and an attacker who inflates F pays it to the miner with no net gain. It is systematic, though: every deposit is off by its fee, and it breaks two documented invariants at once, namely that amount excludes the fee and that a receive carries no fee field.

Only `gettransaction`'s top-level amount and fee are affected; `listtransactions`, `z_listtransactions`, `getreceivedbyaddress`, and `listreceivedbyaddress` use the per-output value and stay correct. Self-transfers are unaffected, since the delta is $-F$ there and the $+F$ cancels to zero.

This was confirmed by reading the `librustzcash` storage path: `store_decrypted_tx` calls `determine_fee` for every decrypted transaction, and for a fully shielded transaction (which every `zecd` receive is) the fee is computed from the public `valueBalance` and written to `transactions.fee`, so a pure receive is stored with `fee_paid = Some(F)`.

Recommendations

Gate the fee add and the `fee` field on the wallet having actually sent the transaction, using the same `account_balance_delta < 0` signal the code already uses elsewhere, so that a pure receive reports `amount = N` with no fee field. Verify that self-send and mixed send-with-self-receive cases remain correct.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [cdb0b54](#).

3.11. The setlabel function accumulates unbounded label storage and reloads the whole table on every history RPC

Target	Label storage (setlabel)		
Category	Code Maturity	Severity	Low
Likelihood	Medium	Impact	Low

Description

The setlabel <address> <label> function stores an address-to-label row in labels.sqlite, a side table next to the wallet database. Labels are zecd's own concept, since librustzcash has none, so the validation is zecd's to do, and the handler only checks that the address is a valid Zcash address. It does not check that the address belongs to the wallet, so foreign addresses are accepted, and there is no limit on the label string length (Bitcoin Core caps labels at 255 characters) nor on the number of rows.

```
// src/rpc/wallet_methods.rs:285
pub(crate) fn setlabel(state: &AppState, wallet: Option<&str>, req:
    &RpcRequest) -> Result<Value, RpcError> {
    // [...]
    if !crate::address::validate(&handle.network, addr).is_valid { //
is_valid only, never is_mine
        return Err(RpcError::invalid_address_or_key(/* [...] */));
    }
    labels::set_label(&handle.dir, addr, &label)?; // no
length or row-count cap
    Ok(Value::Null)
}
```

The storage layer just upserts, with no length guard, and nothing ever prunes it. The read path compounds it: labels::all loads the entire table with no LIMIT, and it runs at the top of nearly every history RPC (listtransactions, gettransaction, getaddressesbylabel, listsinceblock, getreceivedby*, and z_listtransactions).

```
// src/wallet/labels.rs:50
let mut stmt = conn.prepare("SELECT address, label FROM
    address_labels"); // no LIMIT
// materialized into a HashMap on every history RPC
```

Impact

An authenticated caller can loop `setLabel` over many freshly generated valid addresses, each with a multimegabyte label; each call is one row, and the body limit only bounds a single call, not the accumulated table. So `labels.sqlite` grows without bound on disk, fully attacker-controlled. This is compounded by the fact that every subsequent `listtransactions`, `gettransaction`, or `getaddressesbylabel` reloads that whole poisoned table into a `HashMap` before doing its actual work, so the cost of every history RPC scales with how much the attacker stuffed in.

There is no impact to funds or keys, and the precondition is an authenticated caller against the operator's own node, which is why this is Low, but it is a clean instance of a missing input and resource limit, and the accumulation and amplification matter at exchange scale. (The same side table's `tx_first_seen` is a weaker secondary instance, since rows are created only for transactions the wallet actually decrypts, which is bounded by real on-chain payments.)

Recommendations

Cap the label length at a reasonable bound (for example, Bitcoin Core's 255 characters), and reject longer values. If foreign-address labels are kept for send-side address-book compatibility, cap the number of foreign-address rows, or apply a separate cap to rows whose address is not the wallet's own. Replace the full-table load in `labels::all` and `first_seen_all` with a query scoped to the txids or addresses actually needed (a join or `WHERE address IN (...)`) so that one RPC does not materialize the entire table, and periodically prune old or already mined `tx_first_seen` rows.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [f8c6120](#).

3.12. The RPC password can be passed as a command-line argument, exposing it through process metadata

Target	Config (CLI arguments)		
Category	Code Maturity	Severity	Low
Likelihood	Low	Impact	Low

Description

zecd can take the RPC Basic-auth password as a `--rpcpassword <PASS>` command-line argument. A process's argv is held in cleartext by the OS and is exposed through `/proc/<pid>/cmdline` (readable by other local users under Linux's default `hidepid=0`), through `ps`, through shell history, and through container specs. So a credential that is meant to be secret leaks into process metadata, where another party on the same host can read it.

```
// src/config.rs:429
#[arg(long = "rpcpassword", value_name = "PASS")]
pub rpc_password: Option<String>,
// :658 the CLI value takes precedence over the config-file value
password: cli.rpc_password.clone().or(rpc_file.password),
```

zecd takes the age mnemonic over stdin or an environment variable to avoid shell-history exposure, but the RPC password has no equivalent protection; there is no environment-variable alternative for `--rpcpassword`, and no warning is logged when it is used on the command line.

Impact

An unprivileged local user on a shared host, or a party with read access to a container or pod spec, can recover the RPC credential from argv, which grants RPC access (balance, address, and memo queries, and, on an unlocked wallet, sending). Exposure requires two conditions: the operator must actually pass `--rpcpassword` (rather than using the config file, a cookie, or `rpcauth`), and another party must be able to read argv. The primary packaged deployment is not affected, since the systemd unit runs `zecd --datadir ...` and does not place the password on the command line. So this manifests in manual runs, handwritten Kubernetes manifests, and development environments, which is why both the likelihood and the impact are Low; it is a real credential-exposure footgun rather than an exploitable bug in the default deployment.

Recommendations

Log a startup warning when `--rpcpassword` is supplied, advising that the RPC password is visible through `/proc` and `ps` and pointing to the config file, a cookie, or `rpcauth` instead. Document the config file (mode 0600), the cookie, and the `rpcauth` hashed-credential path as the default options. Optionally, add an environment-variable input path or deprecate the command-line password. This mirrors bitcoind, which warns against `--rpcpassword` on the command line for the same reason.

Remediation

This issue has been acknowledged by Overpay, and fixes were implemented in the following commits:

- [34504e7](#) ↗
- [75e67d6](#) ↗

3.13. Internal anyhow: :Error messages are forwarded verbatim to the RPC client

Target	Wallet RPC methods (src/error.rs, src/server/jsonrpc.rs)		
Category	Coding Mistakes	Severity	Low
Likelihood	High	Impact	Low

Description

Whenever a handler bubbles up an anyhow: :Error, it is converted into an RpcError that simply stringifies the error, which might include sensitive information.

```
// src/error.rs:161

impl From<anyhow::Error> for RpcError {
    fn from(e: anyhow::Error) -> Self {
        RpcError::misc(e.to_string()) /* Convert Rust error message directly
        */
    }
}
```

That message field goes straight into the JSON-RPC response, again with nothing stripped.

```
// src/server/jsonrpc.rs:83

pub fn error(id: Value, err: &RpcError) -> Value {
    json!({
        "result": Value::Null,
        "error": { "code": err.code, "message": err.message }, /* Error
        object is sent to RPC client */
        "id": id,
    })
}
```

Across the codebase, errors are built with .context(...) / .with_context(...) that attach real internal detail (for example, file paths and network endpoints). These details can be wrapped in an RpcError and delivered directly to the RPC client.

Impact

Depending on which internal call fails, an RPC client could see sensitive detail such as the backend node's cookie-file path or its configured host and port. For example, a `getrawtransaction` whose upstream fetch fails returns a `connect to upstream: ...` message that carries the zebra RPC address or cookie-file path from the connection context. This is a form of infrastructure and node fingerprinting that may help an attacker plan further attacks.

Recommendations

Do not let `e.to_string()` of an arbitrary internal error reach the client directly. Log the full internal error server-side, and deliver a generic message without sensitive information to the RPC client.

Remediation

This issue has been acknowledged by Overpay, and fixes were implemented in the following commits:

- [5818c71 ↗](#)
- [67fa910 ↗](#)

3.14. The cached proving-key path fails sends to Sapling recipients

Target	wallet actor, send path		
Category	Coding Mistakes	Severity	Medium
Likelihood	High	Impact	Medium

Description

The Orchard proving-key cache ([spend] cache_proving_key, default true) routes sends through the PCZT prove path instead of the fused create_proposed_transactions. In do_send, the cached path is selected whenever the wallet cannot spend Sapling notes, which for the default Orchard-only pool set is always.

```
// src/wallet/actor.rs:1547
// spend) Sapling notes must use the fused path, which signs every pool.
// Orchard-only
// wallets - the default - keep the cache. (A Sapling *output*, e.g. a Sapling
// recipient,
// is fine on the PCZT path; only Sapling *spends* are the issue [...])
let orchard_keys = if self.enabled_pools.contains(Pool::Sapling) {
    None
} else {
    self.orchard_keys.clone()
};
```

The comment asserts that a Sapling output (a Sapling recipient) is fine on the PCZT path. That is incorrect. prove_sign_store_pczt calls extract_and_store_transaction_from_pczt with sapling_vk = None:

```
// src/wallet/actor.rs:2117
extract_and_store_transaction_from_pczt::<_,
    zcash_client_sqlite::ReceivedNoteId>(
    db, pczt, None, Some(&keys.orchard_vk),
)
```

librustzcash documents that this errors when the PCZT carries a Sapling bundle and no Sapling verifying key is supplied:

```
// zcash_client_backend-0.23.0 src/data_api/wallet.rs:2195
// If `sapling_vk` is `None`, and the PCZT has a Sapling bundle, this function
```

```
// will return an error.
```

A send to a Sapling address is permitted under the default `AllowRevealedRecipients` privacy policy and produces a Sapling output bundle in the proposal. On the default configuration (Orchard-only pool set with `cache_proving_key = true`) that send therefore takes the cached PCZT path, generates the Sapling output proof, and then fails at the extract step.

Impact

On the default configuration, any send whose recipient is a Sapling address fails. The failure occurs after the Orchard proof has been generated, so the actor spends the full proving time and then returns an error; the transaction is neither stored nor broadcast. Fund safety is not affected, since the inputs are not consumed and no partial transaction is produced, but a core operation (spending to a valid Zcash address) is broken for an entire class of recipients under the shipped defaults. An operator can work around it by setting `cache_proving_key = false`, which restores the fused path, at the cost of the per-send key-rebuild the cache was added to avoid.

Recommendations

Build and cache the Sapling verifying key alongside the Orchard one and pass it to `extract_and_store_transaction_from_pczt` whenever the PCZT contains a Sapling bundle, so the cached path handles Sapling outputs. Alternatively, fall back to the fused path when the built proposal contains a Sapling output rather than only when the wallet can spend Sapling notes. Correct the comment at `src/wallet/actor.rs:1547` either way.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [3d4640a](#).

3.15. /readyz reports ready before the wallet has scanned

Target	health server, readiness		
Category	Business Logic	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

The health server exposes /readyz for load balancers and orchestrators to gate traffic. Its default mode is Connected, which reports ready as soon as zecd has connected to zebra and learned the chain tip; it does not require the wallet to have scanned any blocks.

```
// src/config.rs:865 default readiness mode
// (ReadinessMode::Connected is the default)

// src/health.rs:37 Connected arm of wallet_ready
// returns true when st.connected && tip > birthday, with no check on
// fully_scanned
```

Reads answer from the local WalletDb over short-lived connections that bypass the sync actor, so balance and history RPCs return whatever the wallet has scanned so far. In Connected mode the wallet can therefore present as ready while fully_scanned is still None or far behind the tip, so getbalance, getreceivedbyaddress, and listtransactions return empty or stale results that read as authoritative.

Impact

An operator that fronts zecd with a readiness gate using the shipped default routes production traffic to a wallet that has not finished scanning. During a from-birthday restore or a long catch-up, a deposit poller that calls getreceivedbyaddress or getbalance sees a value lower than what has actually landed on-chain, and an integration that credits deposits from that value under-credits or makes an early decision. The effect is an accounting-integrity problem for a deposit-crediting deployment rather than fund loss, and it self-heals once the wallet catches up. The behavior is documented and configurable: the Synced readiness mode (with max_scan_lag) waits for scan progress and is the correct choice for balance-sensitive traffic.

Recommendations

Document prominently that balance-sensitive deployments must set [health] readiness = "synced", and consider making Synced the default so the safe posture is the one an operator gets

without opting in. At minimum, surface the scan lag in the `/status` and `/readyz` payloads so an integration can see that a ready wallet is not yet caught up.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [d0ebf9f](#).

3.16. The single-instance data-directory lock does not span hosts

Target	data-directory lock		
Category	Code Maturity	Severity	Medium
Likelihood	Low	Impact	Low

Description

zecd guards against two processes writing the same wallet by taking an exclusive lock on `<datadir>/ .lock` at startup.

```
// src/lock.rs:44  fmutex try_lock_exclusive -> flock(LOCK_EX | LOCK_NB)
```

The lock is implemented with `flock(2)`. On Linux, `flock` is host-local: over some network filesystems it is emulated per-host and only excludes processes on the same machine, so it does not prevent two zecd instances on different hosts from both acquiring a lock on a shared data directory.

Impact

If the data directory is placed on a shared network filesystem (for example a Kubernetes ReadWriteMany volume) and two zecd pods start on different nodes, both pass the single-instance check and open the same `WalletDb`. Concurrent writers to the SQLite wallet database are exactly what the lock exists to prevent, and can corrupt it or desynchronize note state. The precondition is a shared-datadir deployment, which is an operator choice, so the likelihood is low, but the potential outcome (wallet-database corruption) is why it is worth flagging rather than leaving implicit.

Recommendations

Document that the data directory must be on a local filesystem and that the lock does not span hosts. For deployments that cannot guarantee local storage, add a host-independent guard (for example a lease/record in the database itself, or an external coordination lock) so that cross-host concurrency fails closed rather than silently proceeding.

Remediation

This issue has been acknowledged by Overpay, and a fix was implemented in commit [ef0e9a4](#).

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Initialization trusts the existing contents of the data directory

Only when a `keys.toml` already exists, `zecd init` refuses to reinitialize; it does not require the wallet database itself to be empty. If a `data.sqlite` that already contains an account is present in the wallet directory while no `keys.toml` is, initialization opens that database, adds the operator's account alongside the preexisting one, and the daemon subsequently selects the first account in the database. A preexisting account would therefore be used to derive the wallet's receive addresses.

```
// src/init.rs:75 (re-init is refused only when keys.toml exists, never for a
// non-empty wallet DB)
if WalletStore::exists(&wallet_dir) { // WalletStore::exists checks
    keys.toml only
    return Err(anyhow!(
        "wallet '{}' is already initialized at {}",
        args.wallet, wallet_dir.display()
    ));
}
```

The same gap exists at run time, not only during initialization. On every startup, the daemon selects the first account in the wallet database (`try_select_account` at `src/wallet/actor.rs:452`) without checking that the account's viewing key derives from the configured seed, and it then generates receive addresses from that account's viewing key (`get_new_address` at `src/wallet/actor.rs:1443`). The only consistency check is a `debug_assert_eq!` on the account index in the bootstrap branch (`src/wallet/actor.rs:1182`), which verifies the index rather than the key, is compiled out of release builds and is skipped entirely when the database already contains an account. A data directory that carries a preplanted account whose keys belong to a different seed than `keys.toml` would therefore be run against, so the operator's receive addresses would be derived from the planted account and payments to them would be controlled by whoever holds that account's spending key.

Reaching this state requires an attacker to write a crafted database into the operator's data directory before initialization or startup. That access already implies a significant local compromise of the host, since the same write capability could replace the binary, the configuration, or other secrets. We therefore treat this as a property of the data-directory trust boundary rather than a distinct attack surface. As a defense-in-depth measure, initialization could require that the database contain no accounts before creating one, and the runtime account-selection path could confirm that the configured seed derives the selected account (an enforced check, not a debug assertion, on every startup), so that an unexpected or tampered database state is rejected rather than silently absorbed.

4.2. The safe-by-default security posture is assumed rather than enforced

zecd's security model assumes a particular deployment posture: the RPC server reachable only on localhost or otherwise behind authentication, a trusted colocated zebra reached over plaintext loopback, and an operator who manages the data directory. The defaults align with this posture, as the RPC binds to 127.0.0.1 and the upstream preset resolves to the local node.

```
// src/config.rs:645 (the RPC bind defaults to loopback, but nothing rejects
a non-loopback value)
let bind: IpAddr = cli
    .rpc_bind
    .clone()
    .or(rpc_file.bind)
    .unwrap_or_else(|| "127.0.0.1".to_string())
    .parse()
    .context("parsing rpc bind address")?;
```

The posture is not enforced by configuration validation, however. Binding the RPC to a non-loopback address, pointing zecd at a remote zebra over plaintext, or enabling automatic unlock of an unencrypted wallet are all accepted without any warning. These can be reasonable choices for an informed operator, and authentication still applies in each case, but an operator who deviates from the assumed posture receives no signal that the trust boundary has widened. We raise this as an observation about where the deployment's security assumptions reside so that they are documented and visible rather than implicit.

4.3. The core-dump opt-out variable disables seed hardening for any value, not just the documented one

zecd applies process-wide hardening at startup to keep the in-memory wallet seed from being captured passively: it disables core dumps and marks the process nondumpable, which also blocks ptrace attaches and /proc/<pid>/mem reads by other non-root processes. A debugging escape hatch, the ZECD_ALLOW_CORE_DUMPS environment variable, opts out of this hardening, and the documentation describes enabling it by setting it to 1.

```
// src/hardening.rs:32
pub fn harden_process() {
    if std::env::var_os(ALLOW_CORE_DUMPS_ENV).is_some() { // presence only;
the value is never read
        info!("{ALLOW_CORE_DUMPS_ENV} set; leaving core dumps and ptrace
enabled");
    }
```

```
        return; // any value,
        including "0", skips hardening
    }
    disable_core_dumps();
    set_non_dumpable();
}
```

The check tests only whether the variable is present, not its value, so any value opts out of the hardening, including 0, false, or an empty string. An operator who sets ZECD_ALLOW_CORE_DUMPS=0 intending to keep the hardening on would instead turn it off, leaving the seed exposable through a core dump, a ptrace attach, or a /proc memory read if the process later crashes or a local process inspects it. The default, with the variable unset, is safe and keeps the hardening on, so this only affects an operator who sets the variable to a value they expect to mean "off". Interpreting the value so that only an affirmative setting such as 1 or true opts out would make the variable behave as documented.

4.4. A stalled upstream response can wedge synchronization indefinitely

zecd bounds the header of each zebra RPC with a request time-out, and the wallet actor wraps every unary call (tip, broadcast, mempool subscription, transaction fetch) in an outer time-out. The sync-batch calls (the compact-block range, the tree state, and the subtree roots), however, read the response body without an outer deadline, and the body read itself is not bounded. An upstream connection that delivers the response headers but then stalls the body, such as a half-open TCP connection left by a network partition or a proxy that holds the connection open, would leave the sync step waiting indefinitely. Because the single-writer actor awaits the sync step in-line, a stall there also leaves the wallet's other writes (sends, address generation, locking) queued behind it.

```
// src/chain/zebra.rs:173
let response = tokio::time::timeout(REQUEST_TIMEOUT,
    self.http.request(req)) // bounds the header only
    .await
    // [...]
// src/chain/zebra.rs:184
let body = Limited::new(response.into_body(), MAX_RESPONSE_BYTES) // body
    read has no timeout
    .collect()
    .await
```

Normal operation against a healthy local zebra does not exercise this, and a clean upstream crash closes the connection and recovers. It is an observation about robustness under a stalled or half-open upstream connection. Bounding the body read or wrapping the whole sync operation in a time-out as the unary calls already are would let the daemon recover from such a connection

rather than wait on it.

4.5. Reconnect backoff does not advance after a postconnection failure

zecd advances its exponential reconnect backoff only when a connection attempt itself fails. A failure that occurs after the connection has been established, such as a failed tip refresh or a failed sync step, marks the client as disconnected without advancing the backoff deadline, and a subsequent successful connection resets the backoff. An upstream that accepts connections but then fails on the tip or sync calls, for example a degraded, overloaded, or version-mismatched zebra, would therefore be retried immediately in a tight loop rather than with backoff, because the connection keeps succeeding and only the postconnection step fails.

```
// src/wallet/actor.rs:445 (the backoff advances only when connect() itself
// fails)
if let Err(e) = self.connect().await {
    let delay = self.backoff.next_delay();
    self.reconnect_at = Instant::now() + delay;
    // [...]
}
// src/wallet/actor.rs:558 (a successful connect resets the backoff; a
// post-connection failure
// sets client = None without ever advancing reconnect_at)
self.backoff.reset();
```

This is an observation about behavior against a degraded but reachable upstream. It consumes CPU and log volume and repeatedly contacts the upstream until it recovers, with no effect on funds, keys, or wallet data. Advancing the backoff on every path that marks the client disconnected, rather than only on a failed connection attempt, would pace the retries against such an upstream.

4.6. Unvalidated upstream responses can panic the scanner

zecd passes certain values from zebra's responses to the librustzcash scanner without validating them, and the scanner enforces those values with assertions that panic on violation rather than returning an error. Two cases stand out. The tree-state response carries a height that zecd uses without checking that it matches the height it requested, while the scanner asserts that the supplied chain state is exactly one block below the scan range. Separately, the per-block transaction index is not capped before it is handed to the scanner, whose transaction-index type is a 16-bit integer, so a block with more than 65,536 transactions overflows it. In either case, the per-wallet sync task panics, which permanently stops that wallet's writer until the process is restarted; reads continue to answer in the meantime, which masks the stopped writer.

```
// src/chain/zebra.rs:400 (tree_state uses the reply's height without
    checking it against the request)
height: u64::from(reply.height),
```

```
// src/chain/zebra.rs:559
pub fn block_to_compact(block: &Block, sapling_size: u32, orchard_size: u32)
    -> pb::CompactBlock {
    // [...]
    index: index as u64, // :566 not capped; librustzcash TxIndex is u16
    and panics above 65,536 txs
```

The normal upstream is the operator's own trusted node, so triggering this requires a misbehaving, buggy, interposed, or compromised upstream, and the transaction-count case additionally requires a nonconsensus block that an honest node cannot produce. One height-boundary edge of the tree-state case can also occur with an honest node when a scan begins at the very first block, which happens only where shielded activation is at height one, that is, regtest. Validating these response values before they reach the scanner, as the raw-block path already validates a block's claimed height, and isolating the sync path so that an upstream-induced panic cannot take down the wallet's writer would make this path robust against an unexpected upstream response.

4.7. Through `sync.interval_secs = 0`, the idle delay between poll loops is removed

With no lower bound, `interval_secs` is read straight from config.

```
// src/config.rs
let sync = SyncConfig {
    interval_secs: sync_file.interval_secs.unwrap_or(20), /* No limit on
    interval_secs */
    rebroadcast_secs: sync_file.rebroadcast_secs.unwrap_or(60).max(1),
};
```

```
// src/daemon.rs
sync_interval: Duration::from_secs(config.sync.interval_secs), /* It uses
    config directly */
```

```
// src/wallet/actor.rs
let wait = if self.client.is_some() {
    self.sync_interval
```

```
} else {  
    self.reconnect_at.saturating_duration_since(Instant::now())  
};
```

If `interval_secs` is set to 0, it will keep polling the backend continuously rather than waiting between syncs. This requires an explicit misconfiguration, and its effect is wasted CPU and backend load rather than a correctness or fund-safety issue. To handle this, clamping `interval_secs` with a `.max(1)` or a proper minimum value is enough.

4.8. The SIGTERM does not trigger the same graceful shutdown as Ctrl-C

The `trigger_shutdown()` function is only mapped to SIGINT (Ctrl-C). There is no handler for SIGTERM at all, so a SIGTERM kills the process in the default way (no in-flight request draining, no clean actor or `WalletDb` teardown).

```
// src/daemon.rs  
tokio::spawn(async move {  
    if tokio::signal::ctrl_c().await.is_ok() { /* Ctrl-C is mapped to graceful  
        shutdown as well */  
        info!("received Ctrl-C, shutting down");  
        ctrl_c_state.trigger_shutdown();  
    }  
});
```

However, the documentation describes both SIGINT and SIGTERM as equivalent signals that should trigger a graceful shutdown.

```
// README.md  
Stop a live node with a signal (SIGINT/SIGTERM)
```

```
// docs/OPERATIONS.md  
`zecd stop` (or SIGINT) - graceful: in-flight requests finish, new ones get  
503.
```

Since SIGTERM is the default signal that `docker stop`, `systemctl stop`, and most process managers send, it should be treated as a graceful shutdown. Handling SIGTERM (`tokio::signal::unix::signal(SignalKind::terminate())`) would make the actual behavior match the documentation. However, if the ungraceful shutdown on SIGTERM is intentional, the documentation should be updated instead.

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Component: RPC server, authentication, and request admission

Description

The server module (`src/server/`) exposes the JSON-RPC API over an axum HTTP router in bitcoind's dialect. It terminates HTTP, authenticates every request with HTTP Basic auth (a configured user and password, an `rpcauth` hashed credential, or a generated cookie file when none is set), enforces a work-queue semaphore that bounds in-flight requests and sheds overflow with a 503 like bitcoind, applies a request-body size limit, and frames requests and responses as JSON-RPC 1.0. By default, it binds to `127.0.0.1`.

In the threat model, the RPC client itself is trusted, but the strings it forwards on behalf of its own users (addresses, amounts, memos, and labels) are not, so this layer is where authentication, admission, and the first input checks are enforced before any wallet method runs.

Invariants

- Every request must pass authentication before any wallet method is dispatched.
- Credential comparison must be constant-time and fail-closed, and plaintext credentials must never be persisted.
- The generated cookie credential file must be created with owner-only (0600) permissions.
- The volume of in-flight requests must be bounded so that overload is shed predictably.

Test coverage

Cases covered

- Authentication accept and reject paths, constant-time hash comparison, and a known `rpcauth` vector
- HTTP status mapping (for example, an unknown method to 404, an oversized body to 413, and queue overload to 503)
- JSON-RPC envelope framing and single- and batch-request parsing matching bitcoind's shape

Cases not covered

- Admission of unauthenticated traffic against the work queue, which is gated only after authentication
- Cookie-file permission enforcement when the target path already exists or is a symlink
- Behavior under a concurrent preauthentication request flood

Attack surface

- A preauthentication network client can reach the authentication path. The work-queue bound is applied only after authentication succeeds, so unauthenticated traffic is not counted against it.
- The cookie file holds a complete RPC credential, and its at-rest permissions and symlink handling are enforced only on fresh creation.
- The RPC password can be supplied as a command-line argument, which exposes it through process metadata such as `/proc` and `ps`.
- The default bind is localhost. If it is rebound to a non-loopback address, every method becomes reachable subject only to authentication.
- Internal errors might be stringified and delivered to the RPC client, so internal detail such as file paths can be leaked.
- The `SIGTERM` is not treated as a graceful shutdown, even though the documentation describes both `SIGTERM` and `SIGINT` as methods to shut down gracefully.

5.2. Component: RPC dispatch and wallet method handlers

Description

The `rpc` module (`src/rpc/`) holds the dispatch table and the method handlers, grouped into control, network, blockchain, util, rawtx, and wallet methods. Wallet methods are routed per wallet through the `/wallet/<name>` path, while chain-status methods report synchronization state. Handlers translate bitcoind-dialect parameters into librustzcash operations and format the responses, including deliberately extended behavior such as `validateaddress` reporting a unified address's inner receivers.

Large sends are also exposed through the zcashd-style asynchronous family (`z_sendmany`, `z_getoperationstatus`, and related methods), which returns an operation ID immediately and runs the send on a background task that still funnels through the single-writer wallet actor.

Invariants

- A method routed to `/wallet/<name>` must operate on that wallet rather than the default wallet.
- The `gettransaction.amount` must exclude the network fee, and a fee field must appear only on transactions the wallet actually sent, matching bitcoind semantics.
- User-supplied addresses must be validated and checked against the active network

before use.

- Read RPCs must bound the amount of state they materialize per call.

Test coverage

Cases covered

- Per-RPC integration coverage in the regtest harness (fund, mine, and query flows)
- A bitcoind-client conformance harness that exercises response shapes
- Address parsing and amount handling against zcashd test vectors

Cases not covered

- Chain-status methods issued on a routed nondefault wallet
- The top-level amount and fee of `gettransaction` for a pure shielded receive
- Unbounded label accumulation and the full-table label reload performed on history RPCs

Attack surface

- Addresses, amounts, memos, and labels are attacker-influenced strings that the trusted client forwards on behalf of its users, so their validation and bounds are the handlers' responsibility.
- The `setLabel` function accepts addresses the wallet does not own and imposes no limit on label length or row count, and the history RPCs reload the entire label table on every call.
- The chain-status handlers ignore the routed wallet and report the default wallet's state, which can mislead confirmation logic in a multiwallet integration.
- The `gettransaction` overcredits a pure receive by the network fee and attaches a fee field that bitcoind would only place on a sent transaction.
- The async operation registry's cap only evicts finished entries, so repeated `z_sendmany` calls can queue numerous operations.

5.3. Component: Wallet actor, key material, and storage

Description

Each wallet is a single-writer actor (`src/wallet/`) that serializes all mutating commands over a bounded channel. This serialization is what gives zecd its no-double-spend guarantee: proposal, build, and commit for a send happen inside one closure with no intervening await, so two sends cannot select the same notes. Funds are tracked in a `librustzcash WalletDb` (SQLite), user labels live in a separate `labels.sqlite`, and the mnemonic seed lives in `keys.toml`, optionally age-encrypted and held in memory as a zeroizing secret while the wallet is unlocked.

The `walletpassphrase` unlocks the seed for a bounded time-out, and `walletlock` zeroizes it. Unified spending keys are derived per send and are not cached. The daemon is multiwallet, but only one wallet may hold spending keys; the others are watch-only through a unified full viewing key.

Invariants

- Sends must be serialized so that no two spends select the same notes, preserving the no-double-spend guarantee.
- The decrypted seed must be resident only while the wallet is unlocked and must be zeroized on lock or on time-out expiry.
- A long-running command must not be able to indefinitely block security-critical commands such as `walletlock`.
- At most one wallet in the registry may hold spending authority.

Test coverage

Cases covered

- Send serialization and the balance and note-selection flows in the regtest harness
- The encrypted-wallet unlock and lock cycle, including rejection of an incorrect passphrase
- Age encryption round trips, the encrypted-state flag, and that a view-only wallet stores no seed

Cases not covered

- Actor occupancy under a single very large send, since the recipient count is unbounded
- The latency of `walletlock` while a send is midproof
- The in-memory lifetime of the seed when a send crosses the unlock time-out

Attack surface

- The recipient count per send is unbounded (only ZIP-321's 9999 applies downstream), and proving runs in-line on the serial actor, so a single authenticated send can hold the actor and block `walletlock` for an attacker-chosen duration.
- The label side table accepts unbounded input through the RPC handlers, growing on disk and amplifying every history RPC.
- The seed is decrypted in memory while unlocked. Its lifetime is bounded by lock or time-out, but a long-running send can extend the window during which it stays resident.
- The identity file skips permission and symlink checks once it already exists on disk, so a preplanted symlink or a weak-permission file poses a risk.

5.4. Component: Chain source and the zebra backend

Description

In the reviewed `zebra-only-slim` configuration, `zecd` is a light client that obtains all chain data directly from a `zebrad` over JSON-RPC (`src/chain/` and `src/backend.rs`): compact blocks, commitment-tree state, a synthesized mempool view, `getrawtransaction` lookups, and transaction broadcast. The backend selector (`zebra://host:port`) resolves to an HTTP client that carries the node's Basic credential, taken either from a configured user and password or from the node's cookie file.

The node's verdict on a broadcast is modeled as a `BroadcastOutcome`, whose acceptance is keyed on the returned error code. This node is the chain-source trust boundary: `zecd`'s entire view of balances, confirmations, and incoming payments is whatever the node serves it.

Invariants

- A txid returned from a send must mean the node accepted the transaction and will relay it.
- When the node is not local, the connection must protect the credential and the chain data in transit.
- A response attributed to a queried txid should correspond to that txid.

Test coverage

Cases covered

- Broadcast acceptance and rejection mapping, and transaction fetch returning bytes, height, or not-found, against a fake `zebrad` harness
- Block-range compaction, rejection of height-mismatched or garbage blocks, tree-state and subtree-root decoding, and basic-auth and cookie header construction
- Mempool stream dedupe and close-on-new-block behavior

Cases not covered

- Broadcast error envelopes that carry a message but no numeric code
- Remote (non-localhost) node connections and their transport security
- Verification that a fetched transaction matches the queried txid

Attack surface

- The node is the chain-source trust boundary, so a hostile or misbehaving node or anything on the network path to a remote node can shape the blocks, mempool, and transaction lookups that `zecd` consumes.
- A broadcast error without a numeric code is treated as acceptance, so a transaction the

node rejected can return a success txid to the caller.

- The connection is plaintext with no TLS option, so pointing zecd at a remote node exposes the credential and all chain data in cleartext.

5.5. Component: Background sync engine

Description

The sync engine (`src/sync/`) drives forward synchronization and reorg recovery. It downloads compact blocks into an `FsBlockDb` cache, scans them into the `WalletDb` in batches, and deletes each batch's cached files after scanning. On a chain reorganization, which it detects through `librustzcash` continuity errors, it performs a bounded in-place rewind — first to a requested target below the conflict height then to a shallow bounded target if that one is invalid, and otherwise it returns a terminal error directing a from-mnemonic resync. The error's `safe_rewind_height` hint is deliberately ignored.

Invariants

- The cached block files and their `FsBlockDb` metadata rows must stay consistent with each other.
- A reorg must be recoverable in place, without dropping the client and redownloading the chain.
- Reorg recovery must terminate: when no valid rewind target exists, it must fall back to a from-mnemonic resync rather than loop.
- Reorg handling must reverse and recredit affected notes correctly.

Test coverage

Cases covered

- Forward synchronization and batch scanning in the regtest harness
- An offline reorg unit test that rewinds the wallet and the cache together
- The staged rewind fallback ladder

Cases not covered

- Reorg recovery after the per-batch cache files have been deleted, which the offline test does not exercise because it keeps every file on disk
- A live full-daemon reorg against a running node

Attack surface

- The chain source can present a reorg at any time, so recovery correctness depends on

the cached files and the cache metadata staying consistent.

- After per-batch file deletion, the in-place rewind tries to open files that are already gone and fails, so recovery degrades to a full redownload and the cache metadata table grows without bound over the life of the node.
- And `interval_secs` has no lower bound, so a misconfigured 0 leads to a continuous polling loop instead of the delay between syncs.

6. Assessment Results

During our assessment on the scoped zecd files, we discovered 16 findings. No critical issues were found. One finding was of high impact, nine were of medium impact, and six were of low impact.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.